

env_lib: Networked Multi-Agent Control Environments with Classical and Learning Baselines

Anonymous Authors¹

Abstract

Cooperative multi-agent reinforcement learning is increasingly proposed for networked physical systems such as power grids, vehicle platoons and robot teams, yet many widely used benchmarks are games or particle worlds whose actions, rewards and scales have no physical meaning, and they rarely tell the user how far a learned policy is from the controller an engineer would deploy. We present `env_lib`, a library of 18 Gymnasium environments in eight families of networked multi-agent control problems with continuous states and actions in physical units: power-grid frequency control, cooperative adaptive cruise control, consensus and formation control, oscillator synchronisation, multi-robot localisation and target tracking, cooperative rigid-body physics, and two discrete networking tasks. All environments share one interface with joint per-agent spaces, per-agent rewards and an explicit interaction graph. The continuous environments are written batch-first, so a vector environment advances all its copies with array operations: 2.1 to 3.2 million agent-steps per second on one CPU core, 33 to 43 times the throughput of Gymnasium’s synchronous vector environment. Every environment ships a decentralised classical controller as a reference, and a companion package provides seven multi-agent reinforcement learning baselines (IPPO, MAPPO, MADDPG, MATD3, IQL, VDN, QMIX) with tuned settings and a function that compares a new method with all references on identical seeded episodes.

1. Introduction

Many engineered systems are collections of agents that act on continuous physical quantities and interact through a

network: generators and inverters hold the frequency of a transmission grid, vehicles in a platoon keep a safe gap to their predecessors, and robots share measurements to localise themselves and a target. Their controllers must be decentralised, because each agent senses only its neighbourhood, and robust to disturbances. Multi-agent reinforcement learning (MARL) is a natural candidate for such problems, and the questions a researcher asks are practical: does the learned policy stabilise the system, how much control effort does it spend, and does it beat the classical design?

The standard MARL benchmarks were built for other questions. The particle environments of Lowe et al. (2017), the StarCraft challenges (Samvelyan et al., 2019; Ellis et al., 2023), multi-agent MuJoCo (Peng et al., 2021) and the games collected in PettingZoo (Terry et al., 2021) have game scores as rewards, discrete or normalised actions and no notion of a physical operating point. Domain simulators for power systems (Marot et al., 2020; Wang et al., 2021) or traffic (Wu et al., 2022) are faithful but heavy, each with its own interface. They rarely ship the reference an engineer would compare against: a classical controller for the same task, runnable through the same interface.

`env_lib` fills this gap with a compact, dependency-light library of networked control environments. Its contributions are:

- 18 registered environments in eight families (Table 1, Figure 1), from power-grid swing dynamics and string-unstable vehicle platoons to discrete networking tasks, all with documented dynamics, observation layouts, units and bounds;
- one interface across all of them: Gymnasium’s API with joint spaces of shape `(n_agents, dim)`, per-agent rewards, an explicit interaction graph, and adapters to single-agent libraries and to the PettingZoo parallel API;
- batch-first native vector environments that simulate thousands of copies at once on one CPU core, with Gymnasium’s autoreset semantics;
- a decentralised classical controller for every environment, evaluated against random actions on seeded

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review. Do not distribute.

episodes (Section 5.1);

- seven MARL baselines with tuned settings and a single call that compares a new method with random actions, the classical controller and the learned baselines on identical episodes (Section 5.2).

The library is open source (MIT licence) at (repository link removed for anonymous review), installs with `pip`, and is covered by 1,462 tests and continuous integration on Python 3.9 to 3.13.

2. Related Work

Environment interfaces and suites. Gym and its successor Gymnasium (Brockman et al., 2016; Towers et al., 2024) define the single-agent interface that `env_lib` follows; PettingZoo (Terry et al., 2021) standardises multi-agent interaction through per-agent dictionaries, which `env_lib` supports through an adapter. The multi-agent particle environments (Lowe et al., 2017), SMAC and SMACv2 (Samvelyan et al., 2019; Ellis et al., 2023) and multi-agent MuJoCo (Peng et al., 2021) are the most widely used cooperative benchmarks. VMAS (Bettini et al., 2022) vectorises multi-robot scenarios in PyTorch and JaxMARL (Rutherford et al., 2024) re-implements popular environments in JAX for accelerator-scale training. `env_lib` shares their emphasis on batched simulation but targets control problems, runs on NumPy without an accelerator, and pairs every task with a classical controller.

Domain simulators. Grid2Op and the L2RPN challenges (Marot et al., 2020) address topology control of transmission grids, MAPDN (Wang et al., 2021) voltage control in distribution networks, CityLearn (Vázquez-Canteli et al., 2019) demand response and Flow (Wu et al., 2022) mixed-autonomy traffic. They model their domains in more detail than `env_lib`, whose environments are deliberately compact models, such as the swing equations (Kundur, 1994) or a platoon with actuator lag (Ploeg et al., 2011), that capture the phenomenon a controller must handle and simulate fast enough for many training runs.

Algorithm libraries. EPyMARL (Papoudakis et al., 2021), MARLlib (Hu et al., 2023) and BenchMARL (Bettini et al., 2024) provide many algorithms with configurable experiment pipelines. The baselines of Section 5.2 are deliberately smaller: seven reference implementations written against the `env_lib` interface, with settings tuned for these environments, meant as ready-made points of comparison.

3. The Environment Suite

3.1. A Common Interface

Every environment follows the Gymnasium API (Towers et al., 2024). Observations and actions of multi-agent environments are joint: an observation has shape (n_agents, obs_dim) , one row per agent, and an action $(n_agents, action_dim)$ or $(n_agents,)$. `step` returns the team reward, and the individual rewards are in `info["agent_rewards"]`, so that both cooperative (team) and individual-reward learning are possible without wrappers. `terminated` signals a task outcome (a trip, a collision, synchronisation reached) and `truncated` the environment’s own step limit. All randomness comes from the seeded generator of the environment, so a seed fixes the episode.

Observations are local: each row contains only what the agent could measure or receive from its neighbours. Every networked environment exposes its interaction graph as `env.adjacency`. The features of an observation row are documented, scaled to order one, and listed at run time by the properties `observation_layout` and `observation_scale`. Registered ids come with a catalogue (`env_lib.catalog()` and the `env-lib list` command) that reports agents, spaces, optional requirements and whether a native vector implementation exists. One environment deviates from the joint reward convention for historical reasons: AJLATT-v0 returns per-robot reward and termination arrays; a `TeamReward` wrapper, applied automatically by `env_lib.make_vec`, converts it.

3.2. The Environments

Power-grid frequency control. In `PowerGrid-v0`, bus i of a Kron-reduced, lossless transmission network has angle θ_i and frequency deviation ω_i , governed by the swing equations (Kundur, 1994; Dörfler et al., 2013)

$$\begin{aligned}\dot{\theta}_i &= \omega_i, \\ M_i \dot{\omega}_i &= P_i(t) - D_i \omega_i - \sum_j B_{ij} \sin(\theta_i - \theta_j) + u_i,\end{aligned}\quad (1)$$

with heterogeneous inertia M_i and damping D_i , line susceptances B_{ij} on a small-world graph of 16 buses, step load changes and optional load noise in $P_i(t)$, and a bounded fast frequency response $u_i \in [-u_{\max,i}, u_{\max,i}]$ as the action. Every episode starts from an exact, stable equilibrium with a prescribed line loading. Agents are penalised for frequency deviation, rate of change of frequency and control effort; a frequency beyond 0.5 Hz trips the system and ends the episode. Episodes last 10 s (200 steps).

Vehicle platoon. In `Platoon-v0`, a leader drives an exogenous speed profile (cruising, stop-and-go waves, random changes) and eight followers command their accelerations

Table 1. The environment families (default configuration). “Obs.” is the per-agent observation dimension; “Batch” marks a native vector implementation (the others run in Gymnasium’s synchronous or asynchronous vector environments). The Kuramoto family treats the network as one agent that sets the inputs of all oscillators.

Id	State and dynamics	Agents	Obs.	Action per agent	Interaction graph	Batch	Classical controller
PowerGrid-v0	bus angles and frequencies; swing equations	16	10	power injection	transmission network	✓	droop control
Platoon-v0	gaps, speeds, accelerations; actuator lag	8	13	acceleration command	V2V topology	✓	CACC
Consensus-v0, Formation-v0	planar positions; single or double integrator	8	16	2-D velocity	ring or proximity	✓	Laplacian protocol
KuramotoOscillator-* (4 ids)	phases; coupled oscillators	1	18–75	inputs, coupling gains	coupling matrix	✓	phase feedback
KuramotoOscillatorTorch-* (5 ids)	the same on PyTorch tensors	1	18–75	inputs, coupling gains	coupling matrix	tensors	phase feedback
AJLATT-v0	poses and EKF beliefs; unicycles	4	24	linear, angular velocity	sensing range		encircle target belief
Pistonball-v0	ball and pistons; rigid bodies	20	7	piston velocity	adjacent pistons		ramp towards ball
LineMsg-v0	message states; lossy links	10	3	relay or not	line		always relay
WirelessComm-v0, -v1	packet queues; shared access points	36, 16	18	idle or one of 4 APs	grid		collision-free schedule

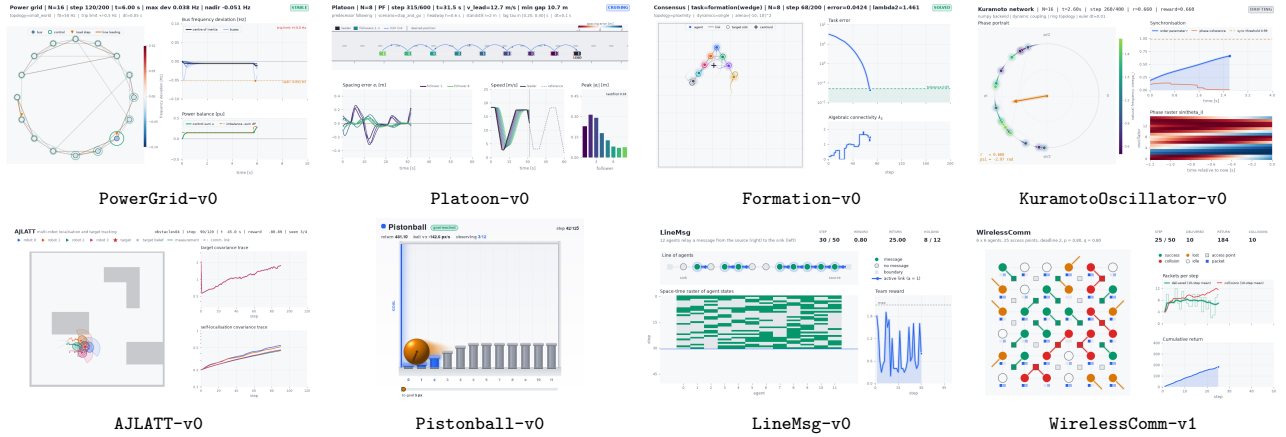


Figure 1. Frames rendered by `env.render()` in “rgb_array” mode. Each frame is a dashboard: the system state next to time series of the quantities that matter for the task (frequencies, spacing errors, disagreement, order parameter, estimation uncertainty, deliveries).

through a first-order actuator lag, $\dot{a}_i = (u_i - a_i)/\tau_i$, discretised exactly. Each follower keeps a constant time headway, $e_i = g_i - (r + hv_i)$, and observes what its vehicle-to-vehicle topology provides (predecessor, predecessor and leader, bidirectional, or sensors only). The environment reproduces the central phenomenon of platoon control, string instability (Swaroop and Hedrick, 1996; Naus et al., 2010): with sensor-only cruise control and a short headway, spacing errors grow along the platoon until vehicles collide. Rewards penalise spacing and speed errors, control and jerk; a collision or a break-up ends the episode (600 steps of 0.1 s).

Consensus and formation. In Consensus-v0 and Formation-v0, eight agents with single- or double-integrator dynamics in a bounded plane must agree on a position (rendezvous) or on a shape, with shifted positions

$y_i = x_i - d_i$ for formation offsets d_i . They communicate over a static graph (a ring by default; line, star, complete or Erdős–Rényi) or a proximity graph that changes as they move, and each agent is penalised by its mean squared disagreement with its neighbours and its control effort (Olfati-Saber and Murray, 2004; Olfati-Saber et al., 2007).

Oscillator synchronisation. In the Kuramoto family, the phases follow the controlled Kuramoto model $\dot{\theta}_i = \omega_i + a_i + c \sum_j K_{ij} \sin(\theta_j - \theta_i)$ (Kuramoto, 1984; Dörfler and Bullo, 2014), integrated with Euler or Runge–Kutta steps. The controller sets the inputs a_i and, in the dynamic coupling mode, the coupling gains. The reward signal is the order parameter, the phase coherence or the frequency-tracking error; synchronisation earns a bonus and, optionally, ends the episode, and a reward mode decides how the signal is paid (Section 4.3). A PyTorch backend simulates batches of

systems on a CPU or GPU.

Multi-robot localisation and target tracking. In AJLATT-v0, four unicycle robots without absolute positioning estimate their own poses and the pose of a moving target with extended Kalman filters, from noisy range-bearing measurements that obstacles on an occupancy map can block, and fuse neighbours' estimates by covariance intersection (Julier and Uhlmann, 1997). The reward penalises the traces of the robots' own and target covariances and collisions, so the team must move to keep the target observed while staying localised; a collision either ends the robot's episode or is only penalised, and a bounded reward mode turns the cost into a positive tracking reward (Section 4.3).

Pistonball. Twenty pistons move a ball to the left wall in a rigid-body simulation (pymunk), a cooperative game popularised by PettingZoo (Terry et al., 2021), here re-implemented with the joint interface, local rewards within a configurable neighbourhood, continuous or discrete actions and a keyboard mode.

Discrete networked tasks. In LineMsg-v0, agents on a line relay a message from a source to a sink over lossy links; relaying keeps a link active, and the team is rewarded for holding and delivering the message. In WirelessComm-v0 and -v1, agents on a 6×6 or 4×4 grid hold packet queues with deadlines and transmit to one of up to four shared access points; a transmission succeeds only if no other agent uses the same access point, in the spirit of the wireless multiple-access problems studied for scalable networked reinforcement learning (Qu et al., 2020). Both expose a credit-assignment problem: in WirelessComm a collision costs the transmitting agent nothing under its own reward.

4. Using the Environments

4.1. Single Environments and Batches

Listing 1 shows the complete workflow. `env_lib.make` creates an environment by id with keyword overrides of any constructor argument; `env_lib.make_vec(id, num_envs)` returns the native batched implementation where one exists and Gymnasium's synchronous vector environment otherwise (`vectorization_mode="async"` selects the asynchronous one). Native vector environments support Gymnasium's three autoreset modes and a reset mask, and every copy reproduces the single environment started from the same state, which the test suite checks.

4.2. Evaluation

`env_lib.evaluate(env, policy, n_episodes, seed)` runs a policy on a single or vector environment and returns the episode returns with their mean, standard deviation and 95 % confidence interval, episode lengths

```
import env_lib

env = env_lib.make("PowerGrid-v0") # kwargs override
obs, info = env.reset(seed=0)      # (16, 10) float32
ctrl = env_lib.baseline_policy(env) # droop control
obs, r, term, trunc, info = env.step(ctrl(obs))
info["agent_rewards"]              # shape (16,)

envs = env_lib.make_vec("PowerGrid-v0", 1024)
res = env_lib.evaluate(envs, ctrl, n_episodes=1024,
                      seed=0)        # about one second
print(res.mean_return, res.ci95)
```

Listing 1. Create, control, batch and evaluate.

Table 2. Reward settings (defaults first). S : reward signal with maximum $S_{\max}$; b : synchronisation bonus; c_i, p_i : tracking cost and collision penalties of robot i .

Setting	Reward or effect
<i>Kuramoto</i> reward_mode	
dense	$S_t + b_t$ at every step
penalty	$S_t - S_{\max} + b_t$: synchronising sooner earns more
progress	$S_t - S_{t-1} + b_t$ (potential-based)
terminal	$S_T + b_T$ at the last step only
sparse	b_t only
terminate_on_sync	synchronisation ends the episode, or is held to the limit
control_cost	$-c \cdot \text{mean}_i a_i^2$ at every step
<i>AJLATT</i> reward_mode	
cost	$-c_i - p_i$
bounded	$e^{-c_i/\sigma} - p_i/\sigma \leq 1$
terminate_on_collision	a collision ends the robot's episode, or is only penalised
collision_termination_penalty	extra penalty when a collision ends it
team_reward_weight	$w: (1 - w)r_i + w\bar{r}$

and per-agent returns. `policy=None` samples uniformly random actions from a seeded copy of the action space. On a vector environment every copy contributes the first episodes it finishes, handling all autoreset modes, so random actions, a controller and a learned policy evaluated with the same seed meet the same initial conditions and disturbances.

4.3. Choosing the Reward

Two families expose their reward design as settings, because a literal maximiser of the original rewards can prefer to end episodes early: the Kuramoto signal is paid at every step until synchronisation ends the episode, and AJLATT pays only negative rewards until a collision ends it. The registered ids keep the original rewards, `env-lib` describe flags the issue, and keyword arguments select the alternatives (Table 2): for example `make("KuramotoOscillator-v0", reward_mode="penalty")` or `make("AJLATT-v0", terminate_on_collision=False)`.

Table 3. Environment steps per second on one CPU core, random actions, automatic resets included (benchmarks/benchmark_vector.py).

Environment	n	1 copy	256 copies		speed-up
			native	Sync	
PowerGrid	16	3.3k	134k	3.1k	43×
Platoon	8	7.1k	295k	7.4k	40×
Consensus	8	7.9k	345k	8.7k	40×
Formation	8	7.9k	313k	8.3k	38×
Kuramoto	10	6.9k	316k	9.6k	33×

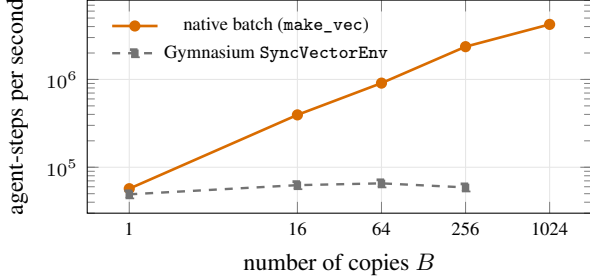


Figure 2. Throughput of Platoon-v0 (8 followers) on one CPU core as a function of the batch size.

4.4. Interoperability and Command Line

Two wrappers connect the joint interface to existing learners: FlattenJointSpaces flattens joint spaces for single-agent libraries, and to_parallel(id) exposes an environment through the PettingZoo parallel API with per-agent dictionaries, which passes PettingZoo’s API test. The env-lib command runs the most common tasks without code:

```
env-lib list --continuous # the catalogue
env-lib describe Platoon-v0 # spaces, layout
env-lib run Formation-v0 --gif run.gif
env-lib evaluate PowerGrid-v0 --episodes 64 \
    --num-envs 64
env-lib bench Platoon-v0 --num-envs 1024
```

4.5. Performance

The continuous environments implement their dynamics once, for arrays with a leading batch axis; the single environment is the batch of one. A vector environment therefore advances all copies with a few array operations instead of a Python loop over copies. Table 3 and Figure 2 report the throughput on one core of an Intel Xeon at 2.8GHz with random actions and automatic resets included: 2.1 to 3.2 million agent-steps per second at 256 copies, 33 to 43 times Gymnasium’s SyncVectorEnv, whose throughput stays at a few thousand steps per second whatever the batch size. Rendering is independent of simulation and only runs when requested.

 Table 4. Mean return of uniformly random actions and the classical controller on the same 64 seeded episodes (one per copy of a 64-copy vector environment, seed 0; benchmarks/benchmark_baselines.py). Higher is better. [†]Original reward; episode length in parentheses.

Environment	Controller	Random	Controller
PowerGrid	droop	−562.1	−0.76
Platoon	CACC	−5,604	−46.3
Consensus	Laplacian	−17,327	−1,414
Formation	Laplacian	−18,166	−1,608
Kuramoto-v0 [†]	phase feedback	24.0 (21)	17.2 (10)
penalty mode	phase feedback	3.22	6.86
Kuramoto freq. sync	phase feedback	−128.1	−112.0
AJLATT [†]	encircle	−627 (16)	−7,090 (115)
no termination	encircle	−34,273	−7,415
bounded mode	encircle	39.9	223.6
Pistonball	ramp	−253.7	801.3
LineMsg	always relay	42.7	95.0
WirelessComm-v0	schedule	388.0	923.8
WirelessComm-v1	schedule	140.5	339.4

5. Integrated Baselines

5.1. Classical Controllers

env_lib.baseline_policy(env) returns a decentralised classical controller for every environment: a pure function of the observation, so the same call drives a single environment or a batch of thousands, and its gains are keyword arguments. The controllers are the standard designs of each field: droop control $u_i = -k\omega_i$ for the grid, cooperative adaptive cruise control $u_i = k_p e_i + k_d(v_{i-1} - v_i) + k_a a_{i-1}$ with string-stable default gains for the platoon (Ploeg et al., 2011), the Laplacian protocol $u_i = k \sum_{j \in \mathcal{N}_i} (y_j - y_i)$ for consensus and formation (Olfati-Saber and Murray, 2004), frequency compensation with mean-field phase feedback for the oscillators, a formation around the target belief for AJLATT, a ramp towards the ball for Pistonball, “always relay” for LineMsg (the optimal policy) and a collision-free access schedule for WirelessComm.

Table 4 compares them with random actions. The two reward designs of Section 4.3 show in the returns: with the original Kuramoto reward the controller, which synchronises in half the steps, collects less than random actions, and in the original AJLATT episode random robots end their negative rewards by colliding early. With reward_mode="penalty" for Kuramoto and without collision termination or with the bounded reward for AJLATT, the controller is clearly ahead; we recommend these settings for training.

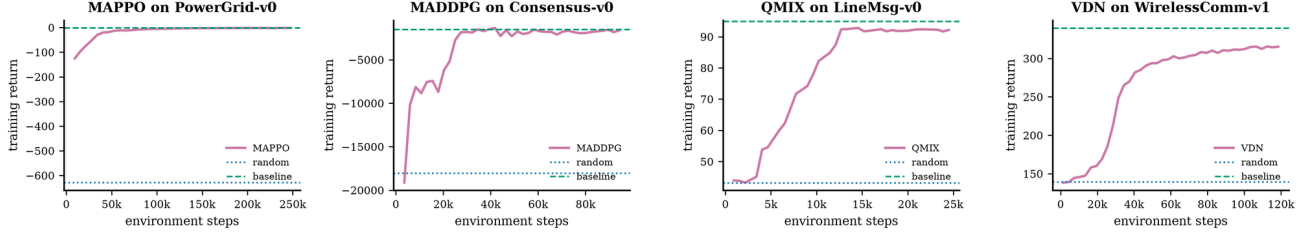


Figure 3. Training return of one method of each family with its preset, against the evaluation returns of random actions and the classical controller (examples/algorithms/marl_training_demo.py).

Table 5. The learning baselines.

Method	Family	Idea
IPPO	on-policy	PPO per agent, own reward
MAPPO	on-policy	centralised critic $V(s, i)$
MADDPG	off-policy	deterministic actors, critics $Q_i(s, a)$
MATD3	off-policy	twin critics, target smoothing
IQL	value-based	independent DQN per agent
VDN	value-based	$Q_{\text{tot}} = \sum_i Q_i$
QMIX	value-based	monotonic mixing conditioned on s

5.2. Learning Baselines

The companion package `marl_algorithms` implements seven widely used cooperative MARL methods against one small core (Table 5): independent and multi-agent PPO (Schulman et al., 2017; Schroeder de Witt et al., 2020; Yu et al., 2022) with generalised advantage estimation (Schulman et al., 2016), MADDPG and MATD3 (Lowe et al., 2017; Ackermann et al., 2019) with centralised critics, and independent Q-learning, VDN and QMIX (Tan, 1993; Mnih et al., 2015; Sunehag et al., 2018; Rashid et al., 2018). All collect experience from the native vector environments, act from local observations (centralised information is used only in training), share or separate parameters across agents, save and load, and evaluate through `env_lib.evaluate`. Standard simplifications are stated in the code: feed-forward networks, a transition replay for the value-based methods, and the concatenated observations as the global state.

Seventeen tuned presets pair the methods with the environments whose action type they support; each trains in under two minutes on one CPU core. Table 6 lists representative results and Figure 3 learning curves (all 17 in Appendix B). Every learned policy but one closes at least 98 % of the gap between random actions and the classical controller; the exception, QMIX on WirelessComm, closes 90 %. All methods match the optimal relay on LineMsg, and VDN comes within 0.5 % of the collision-free schedule on WirelessComm; on the continuous tasks the classical controllers remain ahead, which makes these environments a demanding test for new methods. Independent Q-learning does not learn WirelessComm at all (152 after the VDN budget): under per-agent rewards a collision costs the sender nothing,

Table 6. Tuned presets: mean return of the trained deterministic policy on 64 seeded evaluation episodes (training seed 0, evaluation seed 1); each trained in 16 to 106 s of CPU time on one core. Higher is better.

Method	Environment	Random	Trained	Controller
MAPPO	PowerGrid	-629.3	-0.89	-0.79
MAPPO	Platoon	-5,350	-65.0	-46.4
IPPO	Consensus	-18,041	-1,550	-1,518
MADDPG	Consensus	-18,041	-1,681	-1,518
VDN	WirelessComm-v1	139.2	338.0	339.6
QMIX	LineMsg	43.2	95.0	95.0

```
from marl_algorithms import compare

def my_method(obs): # (num_envs, n_agents, obs_dim)
    ...             # -> (num_envs, n_agents) actions

report = compare("PowerGrid-v0", ["mappo", "ippo"],
                seeds=(0, 1),
                policies={"mine": my_method})
print(report) # aligned table
report.to_csv("power_grid.csv") # or .to_markdown()
mappo = report.algorithms[("mappo", 0)] # reusable
```

Listing 2. Comparing a proposed method with the baselines.

while VDN and QMIX, trained on the team reward, learn to leave contested access points to one agent.

5.3. Comparing a New Method with the Baselines

A new method is judged against all three kinds of reference. The function `marl_algorithms.compare` does this in one call (Listing 2): it trains the learning baselines, by default every method with a preset for the environment, over the requested seeds, and evaluates them together with random actions, the classical controller and the user’s policies. Every policy is evaluated on the same episodes: the copies of the evaluation environment are reset with the evaluation seed and only the first episode of each copy counts (chunks of 64 episodes with consecutive seeds beyond 64 episodes), so that automatic resets after episodes of policy-dependent length cannot give different policies different episodes. All arguments are checked before any training starts. The report prints as a table, exports to CSV and Markdown, and keeps the trained baselines for reuse.

Figure 4 shows the result for a proposed distributed con-

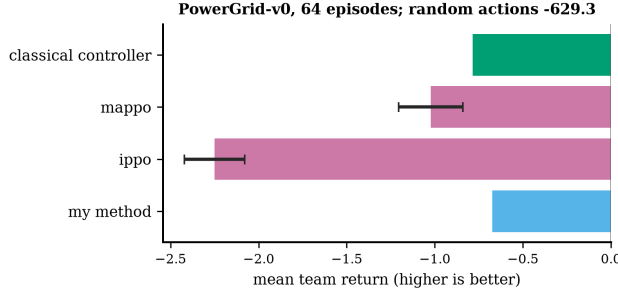


Figure 4. Output of `examples/algorithms/baseline_comparison.py`: mean return on 64 shared evaluation episodes (seed 1); error bars are the standard deviation over two training seeds.

troller on PowerGrid-v0: droop control that also reacts to the neighbours’ mean frequency deviation, $u_i = -k(\omega_i + \beta \bar{\omega}_{N_i})$. It improves on droop control (-0.675 against -0.786), MAPPO and IPPO trained over two seeds reach -1.02 ± 0.18 and -2.25 ± 0.17 , and random actions -629.3 . The same comparison runs from the shell as `marl-train compare PowerGrid-v0 --seeds 0 1 2`.

6. Engineering and Reproducibility

The distribution installs with `pip` from the repository, supports Python 3.9 to 3.13 and needs only NumPy, SciPy, matplotlib, Gymnasium and PyYAML; PyTorch, pymunk/pygame and video export are optional extras loaded on first use. 1,462 tests cover the Gymnasium contract of every environment, the equivalence of single and batched simulation, seeded determinism, the controllers, the wrappers and the learning baselines, and run in continuous integration on Python 3.9 to 3.13 together with smoke tests of the examples. Rendering works headless, episodes export to GIF, MP4 or `.npz` trajectories, and a handbook documents the model equations, observation layouts and parameters of every environment. The distribution also contains small research utilities (publication-quality plotting, PyTorch building blocks and experiment parameter management) that are outside the scope of this paper.

7. Limitations and Conclusion

The environments are compact models: the grid is lossless and Kron-reduced, the platoon drives on a single lane, and the robots move in the plane. They simulate on the CPU with NumPy (the Kuramoto family also on PyTorch), which suits many small training runs better than accelerator-scale training. The learning baselines are reference implementations with feed-forward networks and presets tuned for short budgets rather than for peak performance.

`env_lib` offers networked control problems with physical meaning, one interface, fast batched simulation, and for

every task both a classical controller and learned baselines to compare against. We hope it helps new MARL methods for networked systems to be measured against the designs that engineers deploy.

References

- Johannes Ackermann, Volker Gabler, Takayuki Osa, and Masashi Sugiyama. Reducing overestimation bias in multi-agent domains using double centralized critics. *arXiv preprint arXiv:1910.01465*, 2019.
- Matteo Bettini, Ryan Kortvelesy, Jan Blumenkamp, and Amanda Prorok. VMAS: A vectorized multi-agent simulator for collective robot learning. In *Proceedings of the International Symposium on Distributed Autonomous Robotic Systems (DARS)*, 2022.
- Matteo Bettini, Amanda Prorok, and Vincent Moens. BenchmarkMARL: Benchmarking multi-agent reinforcement learning. *Journal of Machine Learning Research*, 25, 2024.
- Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI Gym. *arXiv preprint arXiv:1606.01540*, 2016.
- Florian Dörfler and Francesco Bullo. Synchronization in complex networks of phase oscillators: A survey. *Automatica*, 50(6):1539–1564, 2014.
- Florian Dörfler, Michael Chertkov, and Francesco Bullo. Synchronization in complex oscillator networks and smart grids. *Proceedings of the National Academy of Sciences*, 110(6):2005–2010, 2013.
- Benjamin Ellis, Jonathan Cook, Skander Moalla, Mikayel Samvelyan, Mingfei Sun, Anuj Mahajan, Jakob Foerster, and Shimon Whiteson. SMACv2: An improved benchmark for cooperative multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2023.
- Siyi Hu, Yifan Zhong, Minquan Gao, Weixun Wang, Hao Dong, Xiaodan Liang, Zhihui Li, Xiaojun Chang, and Yaodong Yang. MARLlib: A scalable and efficient multi-agent reinforcement learning library. *Journal of Machine Learning Research*, 24, 2023.
- Simon J. Julier and Jeffrey K. Uhlmann. A non-divergent estimation algorithm in the presence of unknown correlations. In *Proceedings of the American Control Conference*, 1997.
- Prabha Kundur. *Power System Stability and Control*. McGraw-Hill, 1994.

- Yoshiki Kuramoto. *Chemical Oscillations, Waves, and Turbulence*. Springer, 1984.
- Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems*, 2017.
- Antoine Marot, Benjamin Donnot, Camilo Romero, Baltazar Donon, Marvin Lerousseau, Luca Veyrin-Forrer, and Isabelle Guyon. Learning to run a power network challenge for training topology controllers. *Electric Power Systems Research*, 189, 2020.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.
- Gerrit J. L. Naus, René P. A. Vugts, Jeroen Ploeg, Marinus J. G. van de Molengraft, and Maarten Steinbuch. String-stable CACC design and experimental validation: A frequency-domain approach. *IEEE Transactions on Vehicular Technology*, 59(9):4268–4279, 2010.
- Reza Olfati-Saber and Richard M. Murray. Consensus problems in networks of agents with switching topology and time-delays. *IEEE Transactions on Automatic Control*, 49(9):1520–1533, 2004.
- Reza Olfati-Saber, J. Alex Fax, and Richard M. Murray. Consensus and cooperation in networked multi-agent systems. *Proceedings of the IEEE*, 95(1):215–233, 2007.
- Georgios Papoudakis, Filippos Christianos, Lukas Schäfer, and Stefano V. Albrecht. Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2021.
- Bei Peng, Tabish Rashid, Christian Schroeder de Witt, Pierre-Alexandre Kamienny, Philip Torr, Wendelin Böhm, and Shimon Whiteson. FACMAC: Factored multi-agent centralised policy gradients. In *Advances in Neural Information Processing Systems*, 2021.
- Jeroen Ploeg, Bart T. M. Scheepers, Ellen van Nunen, Nathan van de Wouw, and Henk Nijmeijer. Design and experimental evaluation of cooperative adaptive cruise control. In *Proceedings of the IEEE International Conference on Intelligent Transportation Systems (ITSC)*, pages 260–265, 2011.
- Guannan Qu, Adam Wierman, and Na Li. Scalable reinforcement learning of localized policies for multi-agent networked systems. In *Proceedings of the Conference on Learning for Dynamics and Control (LADC)*, volume 120 of *Proceedings of Machine Learning Research*, 2020.
- Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2018.
- Alexander Rutherford, Benjamin Ellis, Matteo Gallici, Jonathan Cook, Andrei Lupu, Garðar Ingvarsson, Timon Willi, Akbir Khan, Christian Schroeder de Witt, Alexandra Souly, et al. JaxMARL: Multi-agent RL environments and algorithms in JAX. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024.
- Mikayel Samvelyan, Tabish Rashid, Christian Schroeder de Witt, Gregory Farquhar, Nantas Nardelli, Tim G. J. Rudner, Chia-Man Hung, Philip H. S. Torr, Jakob Foerster, and Shimon Whiteson. The StarCraft multi-agent challenge. In *Proceedings of the International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, 2019.
- Christian Schroeder de Witt, Tarun Gupta, Denys Makovychuk, Viktor Makovychuk, Philip H. S. Torr, Mingfei Sun, and Shimon Whiteson. Is independent learning all you need in the StarCraft multi-agent challenge? *arXiv preprint arXiv:2011.09533*, 2020.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Peter Sunehag, Guy Lever, Audrunas Gruslys, Wojciech Marian Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z. Leibo, Karl Tuyls, and Thore Graepel. Value-decomposition networks for cooperative multi-agent learning based on team reward. In *Proceedings of the International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, 2018.
- D. Swaroop and J. K. Hedrick. String stability of interconnected systems. *IEEE Transactions on Automatic Control*, 41(3):349–357, 1996.
- Ming Tan. Multi-agent reinforcement learning: Independent vs. cooperative agents. In *Proceedings of the International Conference on Machine Learning (ICML)*, 1993.

J. K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, Niall Williams, Yashas Lokesh, and Praveen Ravi. PettingZoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, 2021.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

José R. Vázquez-Canteli, Jérôme Kämpf, Gregor Henze, and Zoltan Nagy. CityLearn v1.0: An OpenAI gym environment for demand response with deep reinforcement learning. In *Proceedings of the ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys)*, 2019.

Jianhong Wang, Wangkun Xu, Yunjie Gu, Wenbin Song, and Tim C. Green. Multi-agent reinforcement learning for active voltage control on power distribution networks. In *Advances in Neural Information Processing Systems*, 2021.

Cathy Wu, Abdul Rahman Kreidieh, Kanaad Parvate, Eugene Vinitsky, and Alexandre M. Bayen. Flow: A modular learning framework for mixed autonomy traffic. *IEEE Transactions on Robotics*, 38(2), 2022.

Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative multi-agent games. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2022.

A. Observation Layouts and Actions

Table 7. Per-agent observation rows and actions of the continuous environments (`env.observation_layout` gives the column of every feature; all features are scaled to order one).

Environment	Observation row of agent i	Action of agent i
PowerGrid-v0	frequency deviation, rate of change of frequency, disturbance, exported power, previous action, mean and maximum neighbour frequency deviation, mean neighbour action, capacity, inertia	power injection in $[-u_{\max,i}, u_{\max,i}]$
Platoon-v0	spacing error, relative speed, speed, acceleration, and the communicated predecessor, leader and follower quantities available under the V2V topology (with availability flags)	acceleration command in $[a_{\min}, a_{\max}]$
Consensus-v0, Formation-v0 AJLATT-v0	own position, velocity and formation offset, and the offset-corrected relative positions of the neighbours in a fixed number of slots target belief relative to the robot (position, heading, velocity) and its covariance trace, neighbour robots, the closest obstacle, own pose belief and its covariance trace	velocity or acceleration in $[-u_{\max}, u_{\max}]^2$ linear velocity in $[0, 2]$ m/s, angular velocity in $[-\pi/4, \pi/4]$

B. All Presets

Table 8. All 17 presets (`benchmarks/benchmark_marl.py`; training seed 0, evaluation on 64 episodes with seed 1, CPU seconds of training on one core).

Method	Environment	Env steps	Time [s]	Random	Trained	Controller
IPPO	PowerGrid-v0	200,704	69	-629.3	-2.37	-0.79
IPPO	Platoon-v0	450,560	88	-5,350	-84.8	-46.4
IPPO	Consensus-v0	401,408	83	-18,041	-1,550	-1,518
IPPO	LineMsg-v0	100,800	26	43.2	95.0	95.0
MAPPO	PowerGrid-v0	251,904	106	-629.3	-0.89	-0.79
MAPPO	Platoon-v0	450,560	102	-5,350	-65.0	-46.4
MAPPO	Consensus-v0	401,408	96	-18,041	-1,586	-1,518
MAPPO	LineMsg-v0	100,800	26	43.2	95.0	95.0
MADDPG	PowerGrid-v0	64,000	70	-629.3	-5.69	-0.79
MADDPG	Consensus-v0	96,000	72	-18,041	-1,681	-1,518
MATD3	PowerGrid-v0	64,000	87	-629.3	-5.63	-0.79
MATD3	Consensus-v0	96,000	83	-18,041	-1,737	-1,518
IQL	LineMsg-v0	25,008	9	43.2	95.0	95.0
VDN	LineMsg-v0	25,008	10	43.2	95.0	95.0
VDN	WirelessComm-v1	120,000	74	139.2	338.0	339.6
QMIX	LineMsg-v0	25,008	16	43.2	95.0	95.0
QMIX	WirelessComm-v1	100,000	101	139.2	319.1	339.6

C. Reproducing the Numbers

From the root of a clone of the repository ((repository link removed for anonymous review)):

```

pip install -e ".[all]"
python benchmarks/benchmark_vector.py           # Table 3, Figure 2
python benchmarks/benchmark_baselines.py        # Table 4
python benchmarks/benchmark_marl.py --markdown  # Tables 6 and 8
python examples/algorithms/marl_training_demo.py # Figure 3
python examples/algorithms/baseline_comparison.py # Figure 4

```

All numbers were measured on one core of an Intel Xeon at 2.8 GHz with `OMP_NUM_THREADS=1` and one PyTorch thread; training runs are deterministic for a given seed, thread count and software stack.